



Two-Step Ticket Confirmation & Reset Flow Architecture

Pre-Ticket Verification, Context Modification & Cancellation System (TicketX / AutomationX)

System Architecture & Flow Prompt Engineering Specification | Date: August 31, 2026

Architecture & Engineering Specification

1. Background & Problem Statement

✗ Legacy System (Immediate Ticket Creation)

- **Instant creation on first turn:** Bot immediately invoked `create_ticket` upon receiving the first customer report.
- **Junk / Orphaned Tickets:** Typos, wrong module selections, or customer self-resolutions produced invalid tickets on Plane.so.
- **No Review / Reset Mechanism:** Customers had no opportunity to review, correct, or cancel before developer assignment.

🔧 New Architecture (Two-Step Confirmation)

- **Intake & Summarize First:** Extracts module, symptom, and severity, presenting a concise summary for confirmation.
- **Confirmed-only Creation:** Tickets (`TCK-2026-xxxxxx`) are created in PostgreSQL and Plane.so only after explicit customer consent.
- **Full Support for Edit & Reset:** Allows customers to modify details or abort seamlessly without leaving DB clutter.

2. Core Execution Flow

- 1 **Customer Incident Intake:** Customer sends an issue description in LINE (e.g. *"Reporting an issue: Loan Repayment System. Need to revert status from Paid to Pending Payment."*)
- 2 **AI Gatekeeper Analysis:** E.D.I.T.H. extracts entities and flags the action as `CONFIRM_REQUIRED` (No database insertion performed).
- 3 **Summary & Confirmation Prompt:** Bot replies in LINE with a structured summary asking the customer to confirm.
- 4 **Customer Decision Handling:**
 - **Customer Confirms ("Confirm / Yes"):** Executes Sub Flow → Creates `TCK-2026-90928` → Syncs to Plane.so → Returns Ticket ID & SLA.
 - **Customer Modifies ("Change to..."):** Bot updates the issue summary and re-confirms.
 - **Customer Cancels ("Cancel / Fixed already"):** Bot aborts the ticket creation and acknowledges politely with zero DB pollution.

3. Realistic Dialogue Walkthrough

Customer:

Reporting an issue: Loan Repayment System. Need to revert status from Paid to Pending Payment.

Bot (Summary & Confirmation Prompt):

Understood. Here is the summary of your support request:

🚀 **Module/System:** Loan Repayment System

📄 **Details:** Revert status from Paid to Pending Payment

Is this information correct? *(Reply 'Confirm' to submit the ticket or reply with any corrections.)*

👤 **Customer:**

Confirm

🤖 **Bot (Ticket Created Successfully):**

Your request has been registered! Tracking Ticket: **TCK-2026-90928** 📄

Our engineering team will investigate and complete the action within today by 13:19.

4. Conversation State Transition Table

Current State	Customer Input	Gatekeeper Action	Next State	System Behavior
IDLE (Standard)	Reports new bug / outage / error	CONFIRM_REQUIRED	PENDING_CONFIRMATION	Summarize issue and prompt confirmation in LINE (No DB record).
PENDING_CONFIRMATION	"Confirm", "Correct", "Yes", "OK"	CREATE (Confirmed)	TICKET_CREATED	Create ticket in PostgreSQL → Plane.so → Return TCK code.
PENDING_CONFIRMATION	"No", "Change to...", "Typo"	CONFIRM_REQUIRED	PENDING_CONFIRMATION	Update issue context and resend updated summary to re-confirm.
PENDING_CONFIRMATION	"Cancel", "Nevermind", "Resolved"	CANCEL_RESET	IDLE	Abort ticket creation, reset state, and acknowledge politely.
IDLE	Manuals / Policy / FAQ questions	DOCS_INQUIRY	IDLE	Retrieve and answer from project documentation (Sub Flow Docs).
IDLE	Request human agent / manager	HUMAN_REQUEST	HUMAN TAKEOVER	Dispatch <code>human_notify</code> webhook and hand over to human agent.

5. Gatekeeper Prompt Configuration (E.D.I.T.H.)

```
# TWO-STEP TICKET CONFIRMATION & RESET RULES
```

1. NEW INCIDENT INTAKE:

- When customer reports a new bug/outage/issue AND no pending confirmation in history:
- Output ticket_action: "CONFIRM_REQUIRED"
- Extract subject, summary, priority, severity
- The system will summarize and ask the customer to confirm before creating the ticket.

2. CUSTOMER CONFIRMATION:

- When previous message asked for confirmation AND customer confirms ("Confirm", "Correct", "Yes",
- Output ticket_action: "CREATE"
- Output targetAgent: "support"
- Pass accumulated subject and summary to create the ticket.

3. CUSTOMER CORRECTION:

- When customer modifies details ("No, it is ...", "Change to ..."):
- Output ticket_action: "CONFIRM_REQUIRED" with updated summary.

4. CUSTOMER CANCELLATION / RESET:

- When customer says "Cancel", "Nevermind", "Resolved", "Reset":
- Output ticket_action: "NONE"
- Output intent: "CANCEL_RESET"
- Set reply_text to a polite cancellation acknowledgement.

6. Key Business Value

-  **Zero Junk Tickets:** 100% prevention of accidental, unverified, or misdirected ticket creation.
-  **High Data Accuracy:** Pre-validated subsystem names and symptoms save engineering triage time.
-  **Human-Centric UX:** Reassuring, conversational verification that builds user satisfaction.
-  **Clean Plane.so Board:** Developer sprint boards remain clean and free from canceled tickets.